

GestorOps

× Claude Code

Propuesta Técnica de Integración

Arquitectura · Seguridad · Autonomía · Roadmap · Costes
Mayo 2026 · Versión 1.0 · Confidencial

Propuesta técnica completa para integrar Claude Code como motor principal de análisis y mantenimiento técnico dentro del gestor operativo GestorOps. Cubre arquitectura, seguridad, niveles de autonomía, integración con GitHub y roadmap de implementación.

Índice

Parte	Contenido
1	Viabilidad técnica y comparativa Claude Code vs API
2	Arquitectura completa del sistema
3	Niveles de autonomía (1–5)
4	Tipos de tareas — implementación detallada
5	Integración con GitHub (GitHub App)
6	Workers y ejecución aislada
7	Base de datos — esquema completo
8	UX del gestor — diseño de pantallas
9	Seguridad — estrategia y mitigaciones
10	Roadmap de implementación (5 fases)
11	Respuestas a preguntas clave
12	Estimación de costes

Viabilidad Técnica

Veredicto: **SÍ es viable.** Claude Code no es un chatbot — es un agente técnico real con capacidad de leer codebases, ejecutar comandos, modificar archivos y tomar decisiones en múltiples pasos. La complejidad está en la infraestructura alrededor, no en Claude Code.

Capacidades confirmadas de Claude Code

Capacidad	Descripción
Leer codebases completos	Explorar estructura, leer archivos, entender arquitectura
Ejecutar comandos	bash, git, npm, pytest, linters, tests — cualquier CLI
Crear/modificar archivos	Read, Write, Edit con precisión quirúrgica
Decisiones multi-paso	Bucle autónomo hasta resolver el objetivo
Generar informes	Markdown estructurado, JSON para el gestor
Usar herramientas	Grep, Glob, Bash — búsqueda inteligente en código

Limitaciones reales

Limitación	Impacto	Mitigación
Sin contexto infinito	Proyectos >500k tokens son caros	Indexar solo archivos relevantes
No determinista	Dos runs pueden dar resultados distintos	Revisión humana siempre
Sin contexto de negocio	Solo ve código, no lógica implícita	Enriquecer con docs del gestor
Coste por token	Proyectos grandes escalan el coste	Límites + caching + chunking
Sin estado entre runs	No recuerda ejecuciones anteriores	Estado persistido en BD
Rate limits	Límites de Anthropic por organización	Cola de tareas + retry

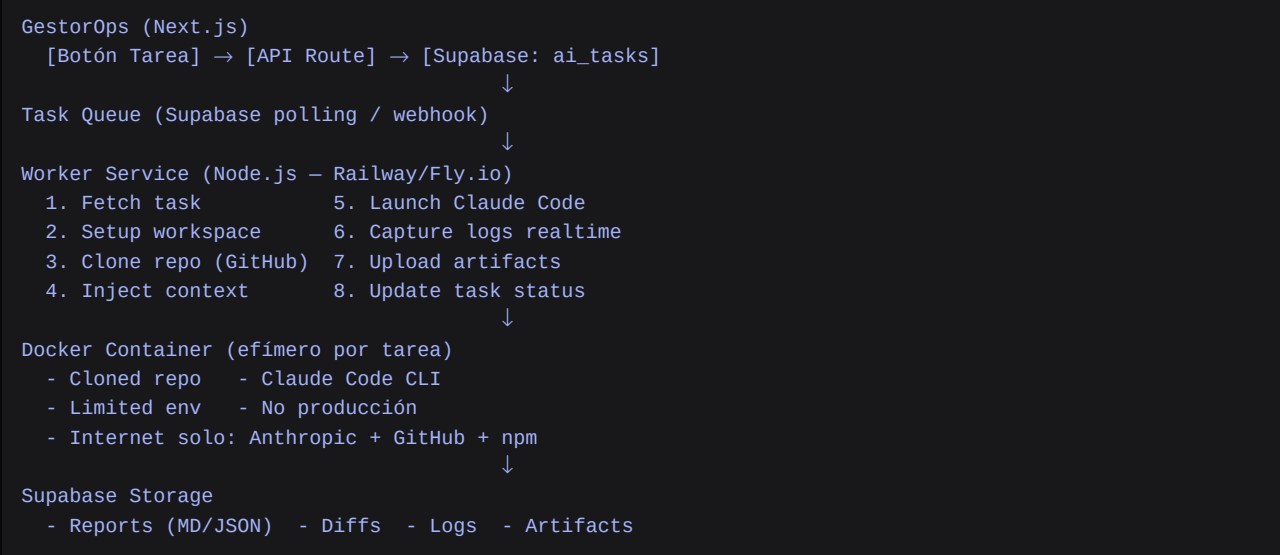
Claude Code vs Claude API — Cuándo usar cada uno

Tarea	Claude Code	Claude API
Analizar repositorio completo	Ideal	Muy costoso
Ejecutar comandos (tests, linters)	Nativo	Imposible
Crear/modificar archivos	Nativo	Manual
Mantenimiento mensual	Ideal	No puede

Tarea	Claude Code	Claude API
Clasificar una incidencia	Overkill	Barato y directo
Chat asistente en el gestor	Overkill	Ideal
Resumir diff de PR	Posible	Más simple

Arquitectura del Sistema

Diagrama de componentes



Flujo completo de una tarea

Paso	Actor	Acción
1	Usuario	Pulsa 'Analizar proyecto' en el gestor
2	Gestor	Verifica permisos, crea ai_tasks (status: queued)
3	Worker	Detecta tarea nueva, marca como 'running'
4	Worker	Crea workspace /tmp/task-{uuid}/, clona repo
5	Worker	Carga contexto: incidencias, notas, último mantenimiento
6	Claude Code	Analiza proyecto según tipo de tarea
7	Claude Code	Genera REPORT.md y PLAN.json en workspace
8	Worker	Sube artefactos a Supabase Storage, actualiza BD
9	Gestor	Muestra resultado al usuario en tiempo real
10	Usuario	Revisa informe, aprueba/rechaza plan
11	Worker	Si aprobado: aplica cambios, abre PR en GitHub
12	Gestor	Registra todo en activity_log y historial técnico

Niveles de Autonomía

NIVEL 1 — READ ONLY

Sin aprobación previa

Claude Code solo lee el proyecto, ejecuta análisis pasivos y genera diagnóstico. Imposible romper nada. Acceso de solo lectura al repositorio.

Puede hacer:

- ✓ Leer cualquier archivo del repo
- ✓ Ejecutar comandos de análisis (git log, grep, tree)
- ✓ Ejecutar tests en modo dry-run
- ✓ Generar informe de diagnóstico

Casos de uso: *Analizar proyecto* · *Auditoría de seguridad* · *Revisar deuda técnica*

NIVEL 2 — PROPOSE

Sin aprobación, resultado para revisar

Claude Code genera plan de acción estructurado, checklist y propuesta de mejoras. El usuario decide si ejecutar el plan.

Puede hacer:

- ✓ Todo el Nivel 1
- ✓ Generar plan de acción en JSON
- ✓ Proponer lista de cambios específicos
- ✓ Estimar impacto, riesgo y horas

Casos de uso: *Mantenimiento mensual* · *Proponer mejoras* · *Análisis de dependencias*

NIVEL 3 — BRANCH

Requiere aprobación explícita

Solo se ejecuta si el usuario ha revisado el plan del Nivel 2 y ha pulsado 'Aprobar'. Modifica código únicamente en rama separada, nunca en main.

Puede hacer:

- ✓ Todo el Nivel 2
- ✓ Crear rama feature/{task-id}-{description}
- ✓ Modificar archivos en esa rama
- ✓ Ejecutar tests automatizados
- ✓ Preparar commit con mensaje descriptivo

Casos de uso: *Preparar parche* · *Actualizar dependencias en rama*

NIVEL 4 — PULL REQUEST

Requiere aprobación + confirmación

Solo si el usuario ha revisado el diff y ha pulsado 'Abrir PR'. Jamás merge automático.

Puede hacer:

- ✓ Todo el Nivel 3
- ✓ git push de la rama al remoto
- ✓ Abrir pull request con descripción técnica
- ✓ Añadir labels y revisores sugeridos

Casos de uso: *Abrir PR para parche aprobado*

NIVEL 5 — AUTO SUPERVISADO

Solo operaciones preautorizadas por admin

Para tareas repetitivas de bajísimo riesgo configuradas explícitamente. NUNCA cambios de lógica, BD, auth o deploys.

Puede hacer:

- ✓ Actualizar dependencias patch version
- ✓ Formatear código según linter
- ✓ Actualizar comentarios en rama separada

Casos de uso: Operaciones rutinarias de bajo riesgo preaprobadas

Tipos de Tareas

Mantenimiento mensual

Nivel 2

1. Revisar dependencias desactualizadas con npm audit
 2. Detectar TODOs/FIXMEs y código complejo
 3. Ejecutar suite de tests y analizar cobertura
 4. Buscar secretos hardcoded
 5. Generar informe con estado general (Saludable/Atención/Crítico)
 6. Plan de acción priorizado por riesgo
-

Revisión de seguridad

Nivel 1

1. Escanear secretos con patrones regex (API keys, tokens)
 2. Ejecutar npm audit / pip-audit con salida JSON
 3. Revisar configuración de seguridad (CORS, headers, auth)
 4. Detectar rutas sin autenticación
 5. Clasificar hallazgos con severidad CVSS
-

Debug de incidencia

Nivel 2-3

1. Leer descripción de la incidencia desde el gestor
 2. Localizar archivos relevantes mediante búsqueda
 3. Trazar flujo de código relacionado con el error
 4. Identificar causa más probable con hipótesis ordenadas
 5. Proponer parche específico (sin aplicar hasta aprobación)
 6. Generar casos de test para validar el fix
-

Actualizar dependencias

Nivel 3-4

1. Detectar paquetes con major/minor/patch updates
 2. Evaluar riesgo de actualización (breaking changes)
 3. Actualizar en rama separada
 4. Ejecutar tests completos para detectar regresiones
 5. Abrir PR con changelog de cambios importantes
-

Generación de documentación

Nivel 2

1. Generar/actualizar README con stack detectado
 2. Documentar endpoints de API automáticamente
 3. Crear changelog desde git history
 4. Generar informe técnico para cliente
-

Onboarding de nuevo proyecto

Nivel 1

1. Analizar repositorio y detectar stack tecnológico
 2. Crear ficha técnica completa para el gestor
 3. Generar checklist inicial de mantenimiento
 4. Registrar dependencias críticas y sus versiones
 5. Detectar problemas de configuración iniciales
-

Integración con GitHub

Recomendación definitiva: GitHub App — no OAuth App. Para un SaaS real con múltiples clientes, la GitHub App ofrece permisos por repositorio, tokens de corta duración (1h) y auditoría por cliente.

GitHub App vs OAuth — Comparativa

Criterio	GitHub App	OAuth App
Permisos por repositorio	Granular por instalación	Todo o nada por usuario
Duración del token	1 hora (installation token)	Larga duración (riesgo)
Actuar como la app	Sí — no como usuario	No — actúa como usuario
Rate limits	5.000 req/h por instalación	5.000 req/h por usuario
Revocación por repo	Sí	No
Auditoría para clientes	Total	Limitada

Permisos mínimos necesarios

```
permissions:
  contents: write      # Clonar repo y crear ramas
  pull_requests: write # Abrir PRs
  issues: read         # Leer issues para contexto
  metadata: read       # Acceso básico al repo
  statuses: write      # Marcar CI checks

# NO necesario:
# admin, org settings, secrets, delete branch
```

Flujo de autenticación segura

1. Instalar GitHub App en repo del cliente
2. Guardar `installation_id` en tabla `repositories`
3. Para cada tarea:
 - a. Generar JWT (válido 10 min) con `GITHUB_APP_ID` + `PRIVATE_KEY`
 - b. `POST /app/installations/{id}/access_tokens`
 - c. Recibir token temporal (válido 1 hora, escopado al repo)
 - d. Pasar token al worker via variable de entorno efímera
 - e. NUNCA guardar el token en base de datos

Workers y Ejecución

Comparativa de plataformas

Plataforma	MVP	Producción	Ventajas	Inconvenientes
Railway	Ideal	Bueno	Deploy simple, Docker nativo, \$5-10/mes	Menos control granular
Fly.io	Bueno	Ideal	Máquinas efímeras por tarea, global, \$0 idle	Curva de aprendizaje
Render	Fácil	Limitado	Configuración trivial	Workers lentos en free
GitHub Actions	Limitado	No recomendado	Ya lo tienes	No es worker general
VPS propio	Complejo	Máximo control	Control total, sin límites	Gestión de infraestructura

Recomendación: Railway para MVP (despliega en 10 minutos). Fly.io para producción (máquinas efímeras por tarea = mejor aislamiento y escalado automático).

Estructura del worker

```
class TaskWorker:
    async run():
        while True:
            task = await fetchNextTask()          # Supabase polling 10s
            if not task: await sleep(10); continue
            await processTask(task)

    async processTask(task):
        workspace = f'/tmp/task-{task.id}'
        try:
            await updateStatus(task.id, 'running')
            await setupWorkspace(workspace, task)
            await cloneRepo(workspace, task)      # GitHub token 1h
            await injectGestorContext(workspace, task) # Incidencias, notas
            result = await runClaudeCode(workspace, task)
            await processResults(workspace, task, result)
            await updateStatus(task.id, 'completed')
        except Exception as e:
            await handleError(task.id, e)
        finally:
            await cleanup(workspace)              # Borrar workspace
```

Base de Datos — Esquema Completo

repositories

Repositorios GitHub vinculados a proyectos del gestor

Campos: `id`, `project_id`, `github_owner`, `github_repo`, `github_url`, `default_branch`, `installation_id`, `last_analyzed_at`, `status`, `metadata`

Sensible: `installation_id` — acceso al repo del cliente

ai_tasks

Cola de tareas para Claude Code

Campos: `id`, `project_id`, `repository_id`, `type`, `level`, `status`, `input_params`, `context`, `result_summary`, `cost_tokens`, `cost_usd`, `timeout_ms`, `incident_id`

Sensible: `input_params`, `context` pueden contener datos del cliente

ai_task_runs

Ejecuciones individuales (con retries)

Campos: `id`, `task_id`, `run_number`, `status`, `worker_id`, `exit_code`, `tokens_input`, `tokens_output`, `started_at`, `finished_at`

Sensible: `workspace_path` solo durante ejecución, nunca persistir

ai_task_logs

Logs en tiempo real (append-only)

Campos: `id`, `run_id`, `level`, `message`, `metadata`, `created_at`

Sensible: Retención: 90 días máximo

ai_task_artifacts

Archivos generados (informes, diffs, patches)

Campos: `id`, `task_id`, `type`, `filename`, `storage_path`, `size_bytes`, `is_sensitive`

Sensible: `is_sensitive=true` para outputs con info de seguridad

approvals

Registro de aprobaciones/rechazos humanos

Campos: `id`, `task_id`, `decision`, `reviewer`, `notes`, `reviewed_at`, `level_approved`

Sensible: Inmutable — auditoría completa

pull_requests

PRs creados por Claude Code

Campos: `id`, `task_id`, `repository_id`, `github_pr_number`, `github_pr_url`, `branch_name`, `title`, `status`

Sensible: Enlace permanente al PR en GitHub

maintenance_runs

Historial de mantenimientos mensuales

Campos: id, maintenance_id, task_id, period, health_score, critical_issues, warnings, report_url

Sensible: health_score 0-100 para dashboards

UX del Gestor

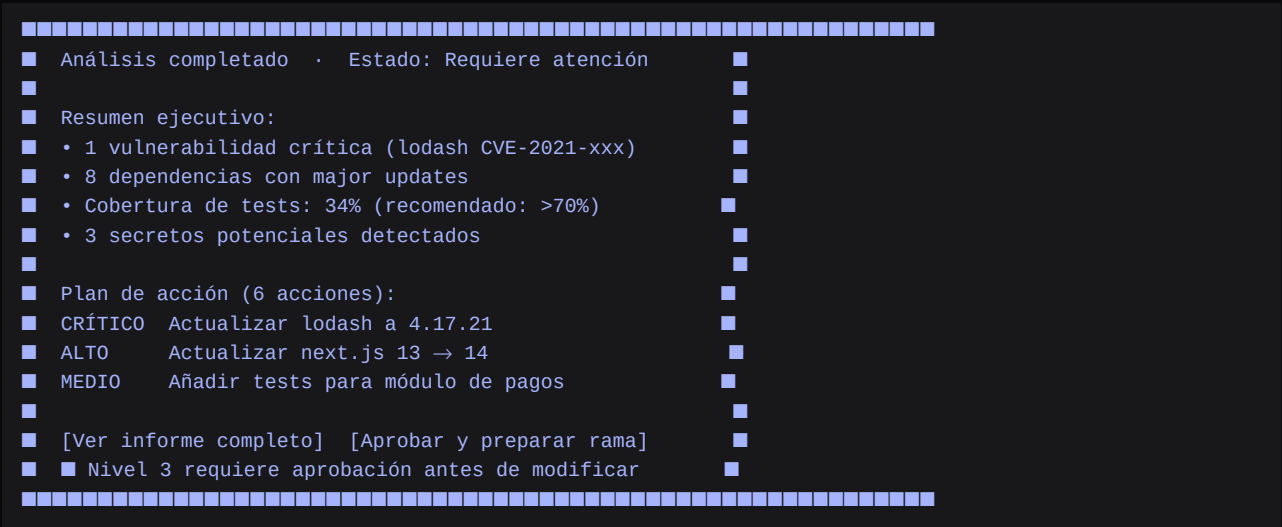
Pantalla de acciones disponibles



Vista durante ejecución



Vista de resultado con aprobación



Seguridad

Claude Code NUNCA debe tener acceso a: producción, bases de datos reales, claves maestras, credenciales críticas, secretos de clientes sin autorización explícita. Esto no es negociable.

Amenaza: Claude Code filtra secretos del cliente

Mitigación: Nunca inyectar credenciales reales en workspace. Solo código fuente. Escaneo previo del repo antes de ejecutar. Outputs revisados antes de guardar.

Amenaza: Se ejecutan comandos destructivos

Mitigación: Lista negra de comandos bloqueados: rm -rf, DROP TABLE, DELETE. Contenedor sin acceso a internet salvo Anthropic/GitHub/npm. Timeout de 10 minutos.

Amenaza: El coste de tokens se dispara

Mitigación: Límite de tokens por tarea (defecto: 50k). Límite mensual por organización. Circuit breaker si se detecta loop. Dashboard de costes en el gestor.

Amenaza: Claude Code rompe el proyecto del cliente

Mitigación: Nivel 1-2: SOLO LECTURA, imposible romper nada. Nivel 3: rama separada nunca toca main. Tests obligatorios. Diff revisado por humano.

Amenaza: Token GitHub comprometido

Mitigación: GitHub App tokens duran 1 hora. Nunca se guardan en BD. Se pasan como env var efímera al contenedor. Rotation automática.

Amenaza: Worker accede a datos de múltiples clientes

Mitigación: Un contenedor por tarea, destruido al terminar. El worker solo recibe datos necesarios para esa tarea. Sin caché entre clientes.

Roadmap de Implementación

FASE 0 — Diseño y Fundación

Duración estimada: 2 semanas

- Diseño del modelo de datos completo
- Crear tablas: repositories, ai_tasks, ai_task_runs, ai_task_logs, ai_task_artifacts
- Configurar GitHub App (permisos mínimos)
- Flujo de conexión de repositorio desde el gestor
- UI: pantalla de repositorios vinculados al proyecto

Entregable: Sin código de agente todavía. Solo infraestructura.

FASE 1 — MVP Análisis Seguro

Duración estimada: 3 semanas

- Worker básico en Railway (Node.js + Docker)
- Cola de tareas con Supabase polling
- Clonar repo con GitHub App token de 1 hora
- Ejecutar Claude Code en modo read-only (Nivel 1)
- Generar informe de análisis básico
- UI: botón Analizar, logs en tiempo real, resultado
- Control de costes y timeouts

Entregable: Entregable: desde el gestor, analizar cualquier repo conectado en ~5 minutos.

FASE 2 — Propuestas con Contexto

Duración estimada: 2 semanas

- Inyección de contexto del gestor (incidencias, notas técnicas)
- Tarea: mantenimiento mensual (Nivel 2)
- Tarea: revisión de seguridad (Nivel 1)
- Plan de acción estructurado en JSON con UI
- Estimación de coste visible antes de lanzar

Entregable: Entregable: mantenimiento mensual completo con plan de acción priorizado.

FASE 3 — Cambios en Rama

Duración estimada: 3 semanas

- UI de aprobación previa para Nivel 3
- Creación de ramas desde el worker
- Aplicación de parches propuestos
- Ejecución de tests automáticos
- Vista de diff en el gestor (similar a GitHub)
- Tarea: debug de incidencia → parche

Entregable: Entregable: desde una incidencia, diagnosticar y preparar parche en rama.

FASE 4 — Pull Requests

Duración estimada: 2 semanas

- Tabla pull_requests en BD
- Apertura de PR vía GitHub API con descripción técnica
- Vista de PRs creados vinculados al gestor
- Registro de aprobación/rechazo en historial

Entregable: Entregable: el gestor abre PRs con descripción técnica generada por Claude Code.

FASE 5 — Automatización Supervisada

Duración estimada: 4 semanas

- Motor de políticas configurables por proyecto
- Tareas periódicas automáticas (cron)
- Nivel 5: operaciones preautorizadas de bajo riesgo
- Dashboard de salud de proyectos
- Reportes automáticos mensuales

Entregable: Entregable: mantenimientos semiautónomos con aprobación en pasos críticos.

Preguntas Clave

— ¿Es viable técnicamente usar Claude Code como motor principal?

Sí. Tiene todas las capacidades necesarias: leer código, ejecutar comandos, modificar archivos, tomar decisiones en bucle. La complejidad está en la infraestructura alrededor, no en Claude Code.

— ¿Qué limitaciones tiene?

Sin contexto de negocio implícito. Coste escala con tamaño del proyecto. No determinista. Sin memoria entre runs. No entiende código legacy sin documentación.

— ¿Claude Code o Claude API — cuándo usar cada uno?

Claude Code para todo lo que toca código real y herramientas. Claude API para clasificación de incidencias, resúmenes de texto, chat asistente en el gestor.

— ¿Qué arquitectura es más segura?

Contenedores efímeros por tarea + GitHub App con tokens de 1h + revisión humana obligatoria para niveles 3+ + sin acceso a producción nunca.

— ¿Qué construir primero?

Conectar repositorios + análisis de solo lectura (Nivel 1). Sin riesgo, valor inmediato, base para todo lo demás. No empezar por los niveles de modificación.

— ¿Qué NO automatizar al principio?

Todo lo que toca producción. Merges de PRs. Despliegues. Cambios de esquema de BD. Modificación de secretos. Cualquier operación irreversible.

— ¿Cómo evitar riesgos legales o de seguridad?

Obtener consentimiento explícito del cliente antes de procesar su código. Revisar políticas de datos de Anthropic API. Para clientes enterprise: evaluar modo on-premise. No usar outputs para entrenar modelos sin autorización.

— ¿Cómo convertir esto en ventaja comercial real?

Diferenciador masivo: 'el único gestor que hace el mantenimiento técnico por ti'. Precio premium justificado. Reduce 60-80% del tiempo operativo de mantenimiento. Upselling natural: más proyectos conectados = más valor generado.

Estimación de Costes

Tipo de tarea	Tokens aprox.	Coste estimado
Análisis de proyecto estándar	10k-20k tokens	~\$0.05 - \$0.08
Mantenimiento mensual completo	20k-80k tokens	~\$0.15 - \$0.30
Debug de incidencia complejo	40k-150k tokens	~\$0.30 - \$0.60
Revisión de seguridad	15k-40k tokens	~\$0.08 - \$0.18
Actualizar dependencias + tests	30k-100k tokens	~\$0.15 - \$0.45
Generar documentación	20k-60k tokens	~\$0.10 - \$0.25

Estimación mensual por agencia con 20 proyectos

Concepto	Cantidad	Coste unitario	Total/mes
Análisis mensuales	20	\$0.08	\$1.60
Mantenimientos mensuales	20	\$0.30	\$6.00
Debug sesiones	10	\$0.60	\$6.00
Revisiones de seguridad	5	\$0.15	\$0.75
Documentación	4	\$0.20	\$0.80
TOTAL IA / MES			~\$15.15

Con un precio SaaS de \$49-99/mes por organización, el coste de IA (~\$15/mes) representa solo el 15-30% del ingreso. Margen operativo excelente. El coste de infraestructura del worker (Railway ~\$10/mes) eleva el total a ~\$25/mes por organización.